La page de connexion Soft-Tech

BTS SIO 1re année — Bloc 1

Activité — champ masqué, bouton « Afficher » et envoi du formulaire

Web, HTML & DOM

Objectifs

- Afficher un mot de passe masqué avec l'inspecteur, puis avec JavaScript.
- Observer où part le mot de passe avec `GET`, puis avec `POST`.
- Expliquer pourquoi les contrôles du navigateur ne protègent rien.

Prérequis

- Balises HTML et attributs, inspecteur du navigateur.
- `querySelector` et `addEventListener` (activité DOM).

Sommaire

Étape	Contenu
Étape 1	Lire le formulaire et l'inspecter (10 min)
Étape 2	Envoyer en GET, puis en POST (10 min)
Étape 3	Programmer le bouton « Afficher » (20 min)
Étape 4	Défi : ce que voit un script (10 min)

Consignes

En binôme, sur poste, 50 minutes. Chacun note ses réponses. Travaillez sans IA : il s'agit de lire quelques lignes et de vérifier ce que fait le navigateur.

> **Règle.** N'utilisez que des mots de passe inventés pour l'exercice, par exemple `Exercice-2026-soft`. Jamais un de vos vrais mots de passe.

Télécharger le projet connexion-soft-tech (ZIP).

Extrayez le dossier, ouvrez-le dans VSCode, puis ouvrez `index.html` dans le navigateur.

Étape 1 — Lire et inspecter

10 minutes.

1. Dans `index.html`, repérez les deux champs. Quelles sont leurs valeurs de `type`, `id` et `name` ?
2. Saisissez une adresse et un mot de passe fictifs. Que voyez-vous dans le champ ?
3. Clic droit sur le champ mot de passe, **Inspecter**. Double-cliquez sur `password` dans `type="password"` et remplacez-le par `text`. Que se passe-t-il ?

À produire : une phrase qui dit ce que `type="password"` protège, et ce qu'il ne protège pas.

Étape 2 — Où part le mot de passe ?

10 minutes.

1. Rechargez la page, remplissez les champs, puis cliquez sur **Se connecter**.
2. Regardez la barre d'adresse. Recopiez ce qui suit le `?`.
3. Dans `index.html`, remplacez `method="get"` par `method="post"`. Enregistrez, rechargez, recommencez. Que contient l'URL maintenant ?

> **Repère**
>
> Avec `POST`, le navigateur peut afficher une erreur : une page ouverte depuis votre disque ne sait pas recevoir de données. C'est normal, regardez seulement l'URL. Un vrai serveur les recevrait.

4. Citez deux endroits où un mot de passe envoyé en `GET` resterait écrit.

À produire : l'URL obtenue en `GET`, et votre réponse à la question 4.

Étape 3 — Le bouton « Afficher »

20 minutes. Le bouton `bouton-oeil` existe déjà dans la page, mais il ne fait rien. Dans `script.js`, recopiez ce code et remplacez les `...` :

```
const champ = document.querySelector("...");
const bouton = document.querySelector("...");

bouton.addEventListener("click", () => {
  if (champ.type === "password") {
    champ.type = "...";
    bouton.textContent = "...";
  } else {
    champ.type = "...";
    bouton.textContent = "...";
  }
});
```

Le `if` teste le type actuel du champ : s'il est masqué, on l'affiche, sinon on le masque. Le texte du bouton doit toujours annoncer ce que fera le prochain clic.

À produire : un bouton qui affiche puis masque le mot de passe à chaque clic, avec le bon texte.

Étape 4 — Défi : ce que voit un script

10 minutes.

1. Saisissez un mot de passe fictif, laissez-le masqué. Ouvrez la **Console** des outils de développement et tapez : `document.querySelector("#mot-de-passe").value`. Qu'obtenez-vous ?
2. Un site charge un script de statistiques hébergé chez une autre entreprise. Que pourrait faire ce script ?
3. Le champ exige 12 caractères (`minlength="12"`). Dans l'inspecteur, supprimez cet attribut et envoyez un mot de passe de 3 caractères. Qui doit vérifier la longueur, finalement ?

À produire : deux phrases : ce qu'un script peut lire, et qui doit faire les vérifications.

Vérification avant de terminer

- [ ] Je sais afficher un mot de passe masqué avec l'inspecteur.
- [ ] J'ai vu le mot de passe dans l'URL avec `GET`, et plus avec `POST`.
- [ ] Mon bouton affiche puis masque le mot de passe, et son texte change.
- [ ] Je sais expliquer pourquoi `POST` ne suffit pas sans HTTPS.

Corrigés

BTS SIO 1re année — Bloc 1

Activité — champ masqué, bouton « Afficher » et envoi du formulaire

Web, HTML & DOM

Étape 1

Champs : `type="email"`, `id` et `name` `identifiant` ; `type="password"`, `id` et `name` `mot-de-passe`. Le passage de `password` à `text` affiche le mot de passe en clair : `type="password"` protège l'écran du regard d'un voisin, pas le secret lui-même.

Étape 2

En `GET`, l'URL se termine par `?identifiant=…&mot-de-passe=Exercice-2026-soft`. En `POST`, l'URL ne contient plus les données : elles voyagent dans le corps de la requête. Un mot de passe en `GET` reste dans l'historique du navigateur, les favoris, les journaux du serveur, ou sur un écran partagé.

Étape 3

```
const champ = document.querySelector("#mot-de-passe");
const bouton = document.querySelector("#bouton-oeil");

bouton.addEventListener("click", () => {
  if (champ.type === "password") {
    champ.type = "text";
    bouton.textContent = "Masquer";
  } else {
    champ.type = "password";
    bouton.textContent = "Afficher";
  }
});
```

Piège fréquent : un `<button>` placé dans un formulaire envoie le formulaire par défaut. Le bouton fourni porte `type="button"` pour l'éviter.

Étape 4

La console affiche le mot de passe en clair : tout script de la page peut le lire, et donc un script tiers peut l'envoyer ailleurs. Les contrôles du navigateur (`minlength`, `required`) se retirent en deux clics : seul le **serveur** peut vérifier de façon fiable.